



Aprendizaje Automático II

Práctica de Aprendizaje por Refuerzo

JAIME CARRERO GALISTEO
LUCAS TIANMING MIGUEL MARTÍN

ÍNDICE

1. Motivación del problema	3
2. Modelado del problema	4
3. Estado del arte	6
4. Soluciones y Diseño Experimental	7
4.1 Monte Carlo	7
4.2 Q-Learning	10
4.3 Deep Q-Learning (DQN)	13
4.4 Mejoras sobre DQN estándar	16
5. Conclusiones y Trabajo Futuro	18
6. Referencias	20

1. Motivación del problema

El juego del Snake es uno de los videojuegos más conocidos mundialmente y además, es uno de los problemas más recurrentes que se suelen utilizar en el Aprendizaje por Refuerzo debido a que a simple vista puede parecer un juego simple, pero en realidad supone un desafío en cuanto a la planificación espacial a largo plazo. A medida que la serpiente obtiene comida del entorno, su cuerpo se expande, alterando la geometría del espacio y aumentando el riesgo de colisión. Desde la perspectiva de la Ciencia de Datos y la Inteligencia Artificial, este entorno proporciona un buen escenario para estudiar cómo un agente puede aprender ese equilibrio entre la recompensa inmediata (la búsqueda de alimento) con la supervivencia a largo plazo.

La propuesta que desarrollamos en este trabajo supone un incremento sustancial de la complejidad del entorno clásico, puesto que en este caso se han introducido dos elementos que añaden un alto componente estocástico: obstáculos dinámicos que varían su posición periódicamente y la aparición de "comida mala" que elimina al agente. Estas modificaciones transforman el juego estático tradicional en un entorno aleatorio que no puede ser resuelto de manera óptima mediante heurísticas fijas o algoritmos de búsqueda tradicionales como A*. La necesidad de esquivar amenazas que cambian de posición en tiempo real obliga a los métodos de Aprendizaje por Refuerzo a generalizar patrones más complejos y a desarrollar políticas capaces de mitigar el riesgo bajo esta incertidumbre.

Más allá de ser un simple videojuego, este problema está ligado y posee aplicaciones directas en el mundo real, como puede ser en sectores de la ingeniería o la logística. La navegación de la serpiente esquivando elementos móviles se asemeja profundamente a los sistemas de conducción autónoma de vehículos o guiado de robots de almacén, los cuales deben calcular rutas óptimas hacia un objetivo mientras esquivan de forma reactiva diferentes obstáculos que pueda encontrar a lo largo del camino. Por otra parte, la gestión del espacio físico se traslada a otros problemas como la gestión de cadenas de suministro automatizadas en los flujos logísticos.

2. Modelado del problema

Para formalizar matemáticamente este problema como un Proceso de Decisión de Markov, vamos a definir a continuación los componentes principales de MDP. Nos basaremos en esta formalización para la implementación del entorno (*snake.py*).

Inicialmente, vamos a definir el **espacio de estados** $\mathcal{S}$ como un espacio continuo acotado con valores dentro del rango $[0.0, 1.0]$. Aunque las variables del juego (coordenadas de la cuadrícula) son discretas, las observaciones que se le proporcionan al agente vendrán normalizadas y vectorizadas para facilitar la convergencia de métodos que tratan de aproximar la función estado-acción como *DQN*. Este vector se compone de 14 variables: las tres primeras variables son binarias e indican la presencia de peligro de colisionar en el frente, izquierda y derecha relativas de la cabeza. Las siguientes cuatro variables tienen codificación *one-hot* y sirven para informar sobre la dirección actual de la serpiente (arriba, derecha, abajo, izquierda). Otras cuatro variables binarias describen la posición relativa de la comida buena respecto de la cabeza del agente en los cuatro ejes, otra variable binaria detecta si hay comida mala en la celda inmediatamente frontal, una variable binaria para la presencia de un obstáculo justo delante; y, finalmente, un valor continuo entre $[0.0, 1.0]$ que representa la longitud actual de la serpiente normalizada respecto del área total de la cuadrícula, calculada como $len(snake)/|Grid|^2$. Dado que este es un espacio continuo, el tamaño del espacio de estados es teóricamente infinito. Para poder aplicar los métodos basados en obtener la tabla Q (*Monte Carlo* y *Q-Learning*), en la implementación de estos algoritmos utilizaremos una función que discretice la variable de la longitud de la serpiente (que es la única verdaderamente continua), en intervalos uniformes. Esto se explicará en la sección correspondiente a la implementación de estos métodos.

En este caso, el **espacio de acciones** $\mathcal{A}$ se trata de un espacio discreto y finito formado por 3 acciones distintas relacionadas con la capacidad de movimiento de la serpiente. Dependiendo de la orientación de la cabeza del agente en cada momento: la acción 0 realiza un giro hacia la izquierda respecto del vector de movimiento actual; la acción 1 ordena mantener la trayectoria en la dirección actual; y la acción 2 hace un giro a la derecha. Esta formulación matemática simplifica el aprendizaje de la política, ya que evita que el agente tenga que aprender de forma implícita la restricción de no poder girar 180 grados sobre sí mismo.

La **función de recompensa** $\mathcal{R}$ ha sido estructurada para guiar al agente hacia el objetivo principal evitando la aparición de comportamientos cíclicos no deseados. Se define formalmente mediante una función por tramos: se asigna una penalización alta de -20.0 si el agente transita a un estado que le lleva a morir como puede ser la colisión contra un muro, contra su propio cuerpo, contra un obstáculo o al consumir comida mala. Por el contrario, se otorga una recompensa positiva de $+10.0$ cuando el agente se alimenta de comida buena. Por otro lado, se introduce un componente que añade $+0.1$ de recompensa si el agente se acerca (en distancia Manhattan) a la comida buena, y -0.1 si se distancia de ella. Adicionalmente, se aplica una penalización por paso constante de -0.01 . Esta elección de recompensas evita comportamientos no deseados y el *reward hacking*, puesto que si el coste por paso no existiera, la serpiente podría oscilar indefinidamente cerca de la

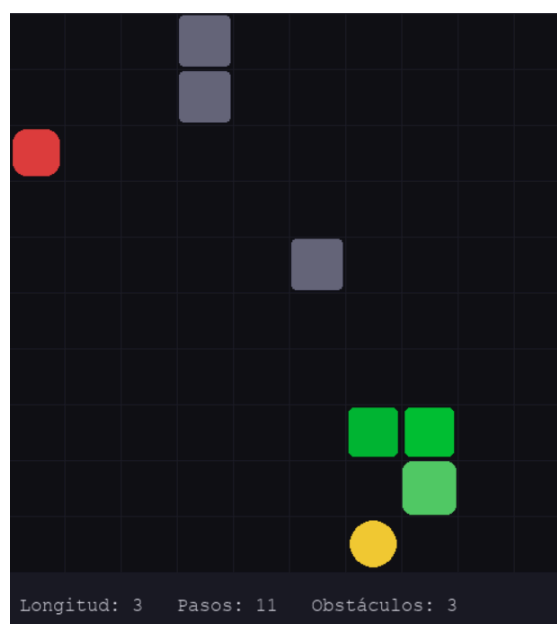
comida para acumular recompensas infinitas de cercanía sin llegar a consumirla, obligando a que la política óptima trace la ruta más corta posible hacia el objetivo.

Sobre la **dinámica de transición** $\mathcal{P}$, podemos decir que este es un entorno estocástico y episódico. La actualización de la posición del agente y su crecimiento son deterministas. Sin embargo, el entorno introduce transiciones estocásticas con la reubicación de la comida buena y mala tras ser consumidas y el movimiento de los obstáculos hacia una de las celdas adyacentes libres. El entorno es episódico, dado que cada episodio cuenta con un estado terminal que finaliza el episodio cuando ocurre una colisión o cuando se alcanza un umbral límite de pasos (*truncamiento*).

El **factor de descuento** se establece en un valor de $\gamma = 0.99$. Este parámetro pondera cuánto importan las recompensas futuras, siendo un valor γ próximo a 1 indicativo de que interesan las recompensas a largo plazo, mientras que un valor próximo a 0 hace al agente más premiar recompensas en el corto plazo. La justificación de este parámetro radica en la necesidad de que el agente planifique a largo plazo y evite priorizar la obtención inmediata de comida ignorando por completo que la trayectoria elegida pudiera conducir al agente a un callejón sin salida o chocar contra los obstáculos móviles en los pasos siguientes.

Finalmente, cabe destacar que el código del entorno ha sido implementado adaptándolo a la API de Gymnasium. Para garantizar la reproducibilidad, el entrenamiento de los agentes se ejecutará bajo la misma configuración preestablecida en el código. El espacio del entorno se basa en una cuadrícula bidimensional de 10x10 celdas y un total de 3 obstáculos dinámicos con una frecuencia de movimiento cada 5 pasos. Asimismo, para evitar bucles infinitos en políticas subóptimas, se fija un límite de 500 pasos por episodio.

La siguiente imagen muestra un ejemplo gráfico del entorno implementado. Las celdas negras se tratan de espacio libre, las grises son los obstáculos dinámicos, las verdes componen el cuerpo de la serpiente, el círculo amarillo es la comida buena y la figura roja consiste en la comida mala.



3. Estado del arte

Como dijimos anteriormente, la resolución del Snake mediante métodos automáticos ha sido objeto de estudio recurrente en el campo de la Inteligencia Artificial, evolucionando significativamente mientras surgían nuevos paradigmas como el *Machine Learning* o recientemente el *Deep Learning*.

La versión clásica y estática del juego ha sido abordada con éxito mediante algoritmos de búsqueda y heurísticas. Como se explica en [1], en estos enfoques, la cuadrícula del juego se modela como un grafo donde cada celda representa un nodo. En el artículo referenciado, se emplea el algoritmo de *búsqueda A** utilizando la distancia de Manhattan como heurística para realizar la ruta más corta desde la cabeza de la serpiente y hasta la manzana. No obstante, estas heurísticas sufren cuando la longitud de la serpiente aumenta, dado que calcular el camino más corto sin fijarse en otros componentes suele llevar a callejones sin salida o a atraparse dentro del propio cuerpo de la serpiente. Para mitigar esto, el artículo combina *A** con comprobaciones de seguridad usando *Branch and Bound*. De esta forma, se fuerza al agente a perseguir su propia cola cuando no existe una ruta segura hacia la comida.

A pesar de la efectividad de las heurísticas en entornos predecibles, la introducción de dinamismo y amenazas móviles vuelve computacionalmente inviables a los algoritmos de búsqueda. Es aquí donde entran en escena las técnicas de Aprendizaje por Refuerzo. Las primeras aproximaciones aplicadas a variaciones del Snake recurrieron a enfoques tabulares como *Q-Learning*. Sin embargo, como detalla [2], surge el problema de la maldición de la dimensión, ya que representar todas las posibles combinaciones de permutaciones de las posiciones del cuerpo y de los muros resulta inmanejable. Para solventar este problema, en este artículo se propone diseñar vectores de observación relativos, codificando el estado no a través de coordenadas globales, sino mediante variables binarias de peligro inmediato en las direcciones respecto de la cabeza del agente y la posición del objetivo. De esta manera, se logra reducir considerablemente el espacio de estados, permitiendo a los algoritmos tabulares converger a políticas efectivas en entornos dinámicos.

En los últimos años, el estado del arte se ha inclinado de forma definitiva hacia las técnicas de *Deep Learning* para resolver las variantes más complejas del juego. Siguiendo el enfoque propuesto en [3], los proyectos más recientes han demostrado que el uso *Deep Q-Networks* permite a los agentes asimilar relaciones más complejas del peligro y generalizar en comportamientos muy eficaces para evadir amenazas en movimiento. Mediante la introducción de memorias de experiencia (*Experience Replay*) y redes objetivo congeladas (*Target Networks*), los agentes basados en *DQN* han conseguido estabilizar el entrenamiento y obtener resultados muy potentes.

4. Soluciones y Diseño Experimental

4.1 Monte Carlo

Descripción del algoritmo

Monte Carlo es un método indirecto basado en valores. El objetivo de dicho método es aprender una función $Q(s, a)$ que relaciona cada par estado-acción con su recompensa esperada, entendida como la suma de las recompensas de elegir esa acción y todas las acciones óptimas posteriores.

Una vez aprendida esta función, la política se deriva de ella seleccionando la acción de mayor valor en cada estado.

La **recompensa acumulada** que el agente busca maximizar se define como

$$R(\tau) = \sum_{i=0}^{\infty} \gamma^i r_{t+i+1} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

donde $\gamma \in [0, 1]$ pondera cuánto importan las recompensas futuras. Un valor γ próximo a 1 indica que interesan las recompensas a largo plazo, mientras que un valor próximo a 0 hace al agente más cortoplacista.

Para actualizar la función Q , Monte Carlo espera a que el episodio termine por completo para calcular el retorno real obtenido. El proceso sigue el bucle de interacción agente-entorno, el agente lee un estado s_t , ejecuta una acción a_t , recibe una recompensa r_t del entorno, y pasa al estado siguiente. Esta secuencia $s_0, a_0, r_1, s_1, a_1, \dots$ se almacena durante el episodio entero. Al final, se recorre al revés para calcular el retorno acumulado G desde cada instante, y se utiliza para actualizar el valor $Q(s, a)$ de cada par visitado mediante una **media iterativa**:

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} (G - Q(s, a))$$

donde $N(s, a)$ es el contador de visitas al par estado-acción.

Discretización del espacio de estados

La función Q ha sido implementada como una **tabla** indexada por estados, esto exige que el espacio de estados sea discreto y finito, sin embargo, la observación del entorno contiene 14 componentes, las 13 primeras son binarias y se mapean directamente a enteros, pero la última (*length_norm*), la longitud normalizada de la serpiente es un valor continuo en $[0,1]$.

Al ser continua, el agente rara vez visitaría exactamente el mismo estado dos veces, lo que impediría que la tabla Q convergiera. Para resolver este problema, se ha discretizado en $N_{\text{bins}} = 10$ intervalos uniformes, transformando cada valor en un índice entero. El estado queda así representado como una tupla de 14 enteros que actúa como clave en la tabla:

```
def discretizar_estado(obs: np.ndarray, n_bins=10) -> tuple:
    estado_discreto = tuple(int(round(x)) for x in obs[:13])
    bin_longitud = min(int(obs[13] * n_bins), n_bins - 1)
    return estado_discreto + (bin_longitud,)
```

Política de exploración: ϵ -greedy

Para seleccionar acciones durante el entrenamiento se emplea la estrategia ϵ -greedy. Una política puramente aleatoria no produce buenos resultados, pero elegir siempre la mejor acción conocida hace que el agente se estanque en mínimos locales. La estrategia ϵ -greedy combina ambas:

- Con probabilidad $\epsilon \rightarrow$ se escoge una acción aleatoria (explorar)
- Con probabilidad $1 - \epsilon \rightarrow$ se escoge la mejor acción según la tabla Q , es decir, $\arg \max_a Q(s, a)$ (explotar)

Diseño experimental

Para estudiar cómo afecta el valor de ϵ a la política aprendida, se han entrenado cinco agentes variando únicamente ese parámetro: $\epsilon \in 0.00, , 0.01, , 0.05, , 0.10, , 0.20$. Dado el elevado coste computacional (500 000 episodios por agente), los experimentos se han paralelizado con *joblib*, aprovechando todos los núcleos del procesador.

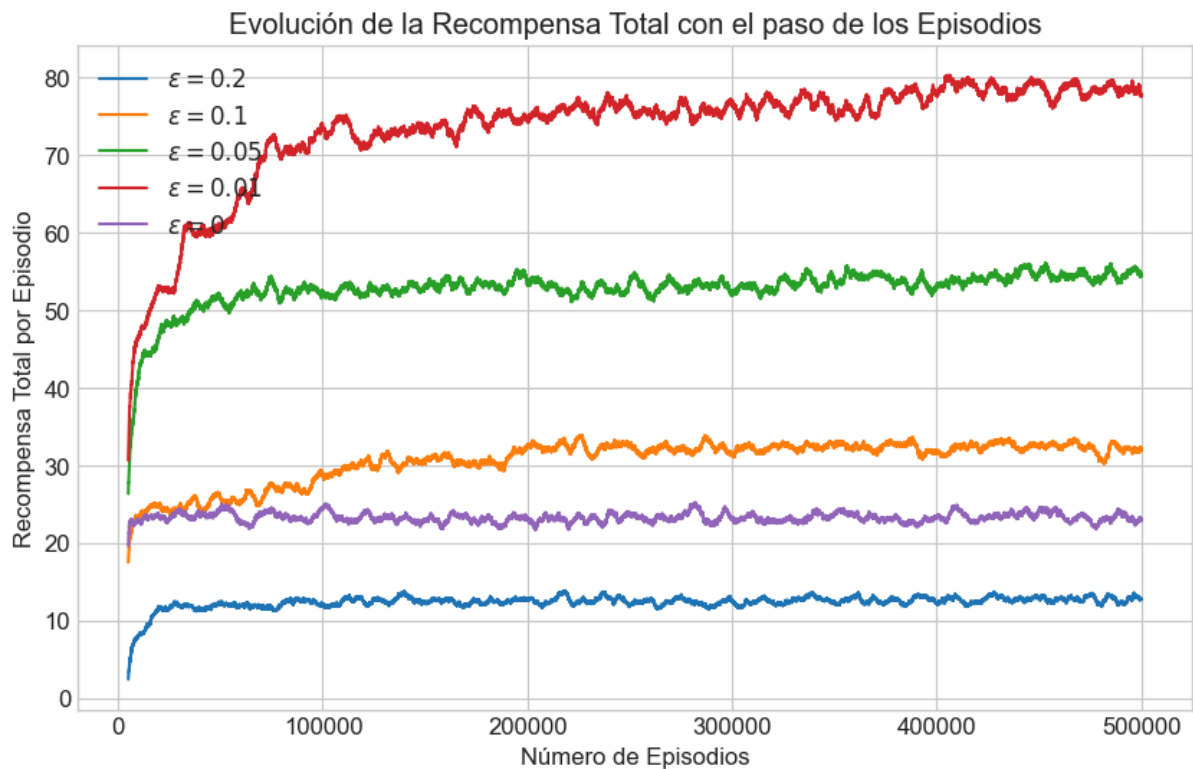
Los hiperparámetros empleados son:

Hiperparámetros	Valor
Número de episodios	500 000
Factor de descuento γ	0.99
Valores de ϵ evaluados	0.00, 0.01, 0.05, 0.10, 0.20
Discretización <i>length_norm</i> (N_{bins})	10
Pasos máximos por episodio	500

El factor $\gamma = 0.99$ se ha fijado próximo a 1 para que al agente le interesen las recompensas a largo plazo, es decir, que la serpiente tome comida buena que es el objetivo.

La evolución de la recompensa acumulada por episodio permite comparar visualmente la velocidad de aprendizaje y el nivel de rendimiento final de cada configuración de ε .

Resultados experimentales



El resultado más destacado es que $\varepsilon = 0.01$ obtiene el mejor rendimiento, alcanzando una recompensa estable en torno a 78-80 puntos. En segundo lugar, $\varepsilon = 0.05$ con un nivel cercano a 54 puntos y le sigue $\varepsilon = 0.1$ en torno a 32. En el extremo inferior se sitúan $\varepsilon = 0.2$ y $\varepsilon = 0$ con recompensas de aproximadamente 13 y 23 puntos respectivamente.

Por ende, podemos sacar como conclusión es que un ε demasiado alto ($\varepsilon = 0.2$) hace que el agente tome demasiadas acciones aleatorias, impidiendo que explote el conocimiento acumulado en la tabla. Por el contrario, un $\varepsilon = 0$ hace que el agente se quede estancado en la política inicial que encuentra. El valor $\varepsilon = 0.01$ consigue los mejores resultados, explota mayoritariamente la política aprendida reservando una mínima exploración.

En cuanto a la velocidad de convergencia, todos los agentes alcanzan su nivel de rendimiento final antes de los 100 000 episodios, manteniéndose estables a partir de ese punto. Esto indica que 500 000 episodios son más que suficientes para que la tabla Q converja bajo este espacio de estados discretizado.

4.2 Q-Learning

Descripción del algoritmo

Al igual que Monte Carlo, Q-Learning es un **método indirecto basado en valores**, aprende una función acción-valor (o función Q) cuya entrada es un estado y una acción, y su salida es la recompensa esperada de ejecutar esa acción y de todas las posteriores. El agente selecciona en cada momento la acción de mayor valor, pudiendo hacer uso de la estrategia ε -greedy para equilibrar exploración y explotación.

La diferencia fundamental respecto a Monte Carlo reside en la forma en la que se actualiza la función Q . Por un lado, en Monte Carlo se espera al final del episodio completo para calcular el retorno real. Por otro lado, Q-Learning **actualiza la tabla en cada paso** de la interacción, tan pronto como recibe la recompensa del entorno.

Dicha actualización se hace usando la fórmula:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \cdot \left(r_t + \gamma \cdot \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right)$$

- α : es la tasa de aprendizaje, indica lo "agresivo" que se está haciendo el entrenamiento.
- r_t : es la recompensa recibida, es decir, la estimación que obtuvimos al actuar en el estado s_t anteriormente.
- $\max_a Q(s_{t+1}, a)$: la recompensa futura estimada (la que vamos aprendiendo).
- γ : el factor de descuento que sube o baja la recompensa futura.
- $Q(s_t, a_t)$: el valor antiguo que se resta para incrementar o disminuir la diferencia en la estimación.

La expresión $r_t + \gamma \cdot \max_a Q(s_{t+1}, a)$ recibe el nombre de objetivo TD, y la diferencia entre dicho objetivo y el valor actual $Q(s_t, a_t)$ es el error TD. La función Q se actualiza de forma iterativa, parte de un valor fijo arbitrario para todos los pares estado-acción y, a medida que el agente explora el entorno, aproxima mejor la recompensa esperada de cada acción en cada estado.

Política de exploración con decaimiento de ε

En contraste con lo realizado en Monte Carlo, para este método se usará un ε con decaimiento multiplicativo por episodio. En este caso, el agente comienza explorando de forma aleatoria ($\varepsilon = 1.0$) y a medida que avanza el entrenamiento, se va reduciendo hasta alcanzar un $\varepsilon_{\min} = 0.01$ (conservamos un 1% de exploración).

La regla de decaimiento es:

$$\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \varepsilon_{\text{decay}})$$

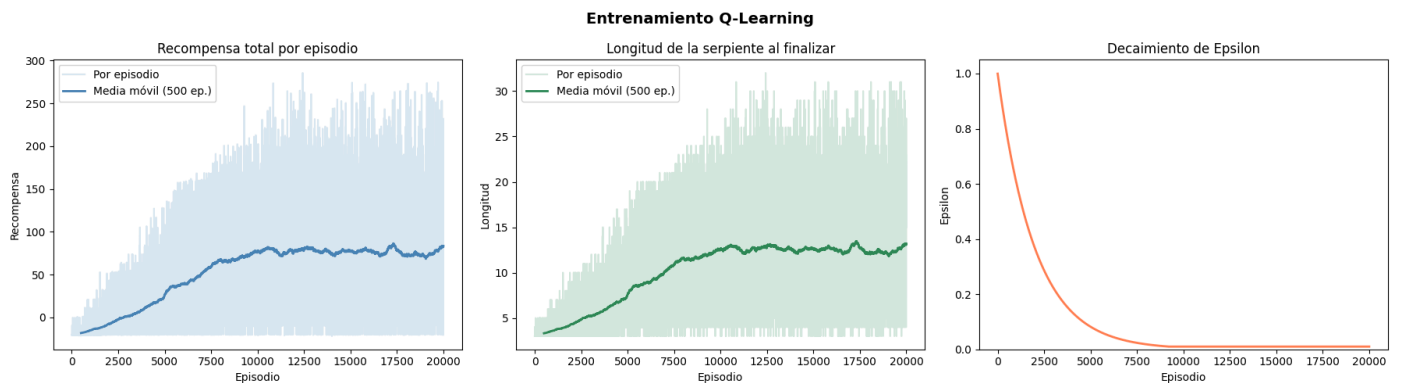
Diseño experimental

Para el entrenamiento, se entrena un único agente con ε decreciente como hemos definido anteriormente en la política de exploración.

Los hiperparámetros empleados son los siguientes:

Hiperparámetros	Valor
Número de episodios	20 000
Tasa de aprendizaje α	0.1
Factor de descuento γ	0.95
ε inicial	1
ε mínimo	0.01
Decaimiento $\varepsilon_{\text{decay}}$	0.9995
Pasos máximos por episodio	500

Resultados experimentales



Para visualizar la evolución del entrenamiento, se presentan tres gráficas: la recompensa acumulada por episodio, la longitud de la serpiente al finalizar cada episodio y el decaimiento de ε a lo largo de los 20 000 episodios de entrenamiento.

La curva de recompensa muestra una tendencia claramente creciente, el agente parte de valores negativos en los primeros episodios. Cuando $\varepsilon \approx 1$ y las acciones son casi aleatorias y va mejorando de forma progresiva hasta estabilizarse en torno a 75-85 puntos a partir del episodio 10 000 aproximadamente. Este comportamiento es coherente con el decaimiento de ε mostrado en la tercera gráfica, la mayor parte de la exploración ocurre en

los primeros 5 000 episodios, momento a partir del cual el agente empieza a explotar de forma creciente la tabla Q .

La gráfica de longitud refuerza esta misma idea, la serpiente parte de una longitud media cercana a 3 celdas y alcanza una media de alrededor de 13 celdas al finalizar el entrenamiento, lo que indica que el agente ha aprendido a comer buena comida de forma consistente.

Finalmente, cabe destacar la alta varianza entre episodios visible en ambas gráficas, algo esperado dado el carácter estocástico del entorno.

4.3 Deep Q-Learning (DQN)

Descripción del algoritmo

Uno de los principales inconvenientes del **Q-Learning clásico es que no escala bien**, el tamaño de la tabla Q crece exponencialmente con el número de estados y acciones, además, el agente no puede generalizar a partir de estados no vistos durante el entrenamiento. Estos problemas se resuelven mezclando el Q-Learning con redes neuronales.

La red neuronal funciona como aproximador de la función Q para cada par estado-acción. Gracias a ello, la red es capaz de aprender una representación compacta de la función Q , generalizar a casos no vistos pero similares a los ya experimentados y escalar.

La función de pérdida que se minimiza durante el entrenamiento es

$$\mathcal{L}(\theta) = \left(r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

donde θ son los pesos de la red que se entrena y θ^- son los pesos de una red auxiliar congelada.

Componentes de la implementación

- **Red neuronal (QNetwork):** la red recibe el vector de observación de 14 componentes y devuelve los valores Q para cada una de las 3 acciones posibles. La red está compuesta por tres capas (una capa de entrada, una capa intermedia y una capa de salida lineal con un valor Q por acción).
- **Búfer de memoria (ReplayBuffer):** en cada paso de la simulación, el agente almacena en el búfer de memoria la transición (s, a, r, s') . Para entrenar la red, en vez de usar la transición más reciente, se muestrea aleatoriamente un batch de transiciones del búfer. Esto rompe la correlación temporal entre experiencias consecutivas y estabiliza el entrenamiento.
- **Red principal y red objetivo (Policy net / Target net):** Para estabilizar el objetivo TD durante el entrenamiento, se utilizan dos redes con la misma arquitectura, la **red principal (Policy net)**, que se entrena en cada paso, y la **red objetivo (Target net)**, cuyos pesos permanecen congelados durante un número fijo de pasos y se sincronizan periódicamente. Sin embargo, la *target net* sigue siendo responsable tanto de seleccionar como de evaluar la mejor acción en s' , lo que produce una sobreestimación de los valores Q .

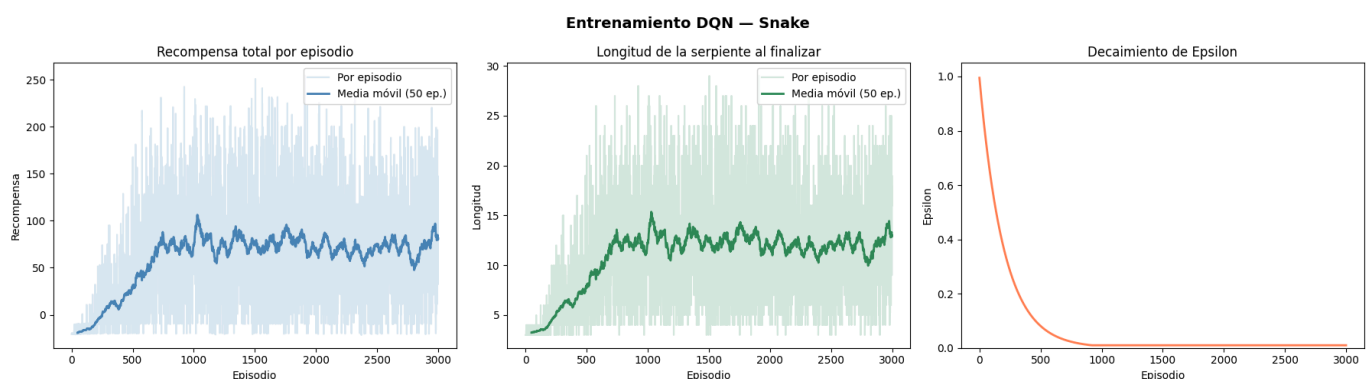
El proceso de entrenamiento sigue los siguientes pasos: tras un arranque en frío para llenar el búfer, en cada episodio el agente elige una acción mediante ϵ -greedy, almacena la transición, muestrea un *batch* del búfer, calcula el objetivo TD con la *target net*, calcula el

MSE entre el valor objetivo y el estimado por la *policy net*, y realiza un paso de retropropagación para minimizar dicho error.

Diseño experimental

Hiperparámetro	Valor
Número de episodios	3 000
Tamaño del <i>batch</i>	128
Factor de descuento γ	0.99
Tasa de aprendizaje	1e-3
ϵ inicial	1
ϵ mínimo	0.01
Decaimiento ϵ_{decay}	0.995
Capacidad del búfer de memoria	10 000
Frecuencia de actualización <i>target net</i>	Cada 10 episodios
Pasos máximos por episodio	500

Resultados experimentales



Las tres figuras muestran la evolución de la recompensa acumulada, la longitud de la serpiente al finalizar cada episodio, y el decaimiento de ϵ a lo largo de los 3 000 episodios de entrenamiento.

La curva de recompensa refleja una tendencia creciente durante la fase de exploración: el agente parte de valores negativos y mejora progresivamente hasta estabilizarse en torno a

75-100 puntos a partir del episodio 1 000. A partir de ese punto la media se mantiene relativamente estable, aunque con una varianza episodio a episodio notablemente alta, algo esperable dado el carácter estocástico del entorno.

La gráfica de la longitud refuerza la misma lectura, la serpiente pasa de una longitud media de 3-4 celdas al inicio a situarse en torno a 12-14 celdas al finalizar el entrenamiento, lo que confirma que la red ha aprendido a comer comida buena de forma consistente y no únicamente a sobrevivir.

Resulta destacable que el DQN alcanza un rendimiento comparable al de Q-Learning. Ambos convergen a recompensas medias similares, pero en tan solo 3 000 episodios frente a los 20 000 necesarios en Q-Learning, por lo que la red hace que el agente generalice a estados no vistos, lo que acelera el aprendizaje.

4.4 Mejoras sobre DQN estándar

Anteriormente se señaló que el DQN tiende empíricamente a sobreestimar los valores Q , debido a que la misma red que selecciona la acción también evalúa su valor, produciendo una sobreestimación sistemática. Para mitigar este problema, hemos implementado dos mejoras que tienen como objetivo mejorar la estabilidad y eficacia del proceso de entrenamiento: **Double DQN** y **Dueling DQN**.

Double DQN

El Double DQN propone separar la selección y la evaluación de una acción en las dos redes ya existentes, la *policy net* y la *target net* en lugar de delegar ambas responsabilidades a la misma red.

La fórmula para el Double Q-learning es

$$r + \gamma \cdot Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{policy}}(s', a'))$$

Comparando con la implementación anterior:

```
# DQN estándar
q_futuros = self.target_net(estados_siguietes).max(1)[0]

# Double DQN
mejores_acciones=self.policy_net(estados_siguietes).argmax(1).unsqueeze(1)
q_futuros = self.target_net(estados_siguietes).gather(1,
mejores_acciones)
```

Esta separación elimina el sesgo de sobreestimación sin añadir coste computacional ni parámetros adicionales al modelo.

Dueling DQN

El Dueling DQN modifica la arquitectura interna de la red para descomponer los valores Q en dos partes distintas:

- **La función de valor** $V(s)$: cuánta recompensa futura se espera obtener desde el estado s , independientemente de la acción tomada.
- **La función de ventaja** $A(s, a)$: cuánto mejor es la acción a respecto a las demás acciones posibles en ese estado.

Combinando ambas se obtienen los valores Q según la expresión:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$$

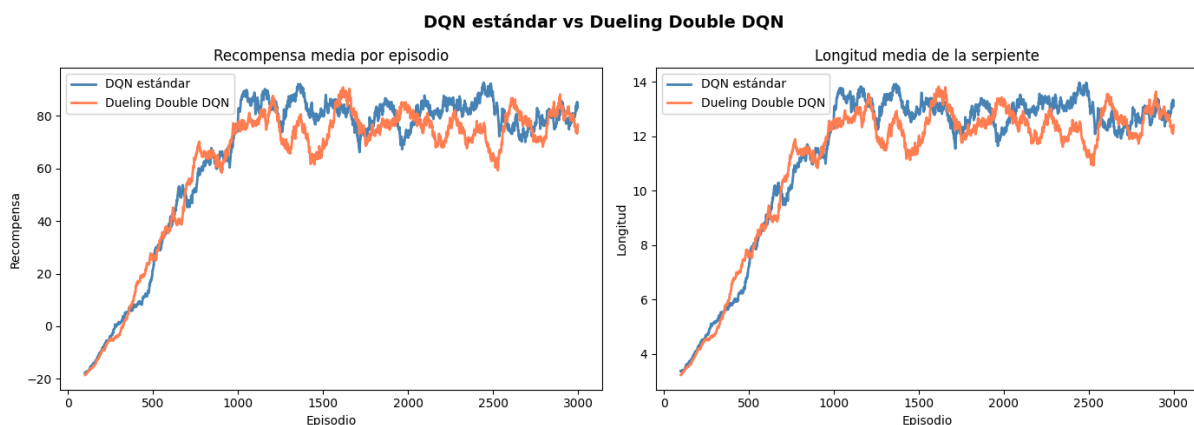
La separación de la media de ventajas garantiza que la descomposición sea única. En la práctica, esto se implementa añadiendo dos cabezas de salida independientes a partir de la capa compartida: una que produce $V(s)$ y otra que produce $A(s, a)$, combinándolas en la capa final.

```
class QNetworkMejorada(nn.Module):
    def __init__(self, n_observaciones: int, n_acciones: int):
        super(QNetworkMejorada, self).__init__()
        self.fc1 = nn.Linear(n_observaciones, 128)
        self.fc2 = nn.Linear(128, 128)
        self.valor = nn.Linear(128, 1)          # Cabeza V(s)
        self.ventaja = nn.Linear(128, n_acciones) # Cabeza A(s,
a)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        v = self.valor(x)
        a = self.ventaja(x)
        return v + a - a.mean(dim=1, keepdim=True) # Q(s,a) =
V(s) + A(s,a) - mean(A)
```

Diseño experimental y resultados

Ambas mejoras se implementan en el agente *AgenteDQNMejorado*, manteniendo los hiperparámetros y número de episodios que el DQN estándar para que la comparación sea completamente justa. La única diferencia es que la capacidad del búfer de reproducción pasa de 10 000 a 50 000 transiciones.



Ambos agentes presentan una fase de aprendizaje prácticamente idéntica, con curvas que ascienden de forma solapada hasta estabilizarse en torno al episodio 1 000, situándose ambos en recompensas medias de 75-85 puntos y longitudes de 12-13 celdas sin que ninguno muestre una ventaja clara. *Double DQN* y *Dueling DQN* mejoran empíricamente la estabilidad del entrenamiento, pero la magnitud de esa mejora depende del entorno, y dado que el espacio de estados es relativamente pequeño, el margen de ganancia es limitado.

5. Conclusiones y Trabajo Futuro

A lo largo de la práctica hemos implementado un total de tres métodos distintos basados en el aprendizaje por refuerzo aplicados al juego del Snake: *Monte Carlo*, *Q-Learning* y *Deep Q-Learning* (con sus variantes *Double* y *Dueling DQN*).

Los tres son métodos indirectos basados en valores cuyo objetivo es aprender una función acción-valor Q que, una vez aprendida, define la política π que maximiza el rendimiento esperado.

Los resultados obtenidos afirman que los agentes han conseguido aprender una política razonablemente buena, todos mejoran claramente respecto a una política aleatoria, aprenden a comer comida buena de forma consistente y evitan colisiones en la mayoría de situaciones, aunque ninguno alcanza un comportamiento óptimo en todas las configuraciones del entorno.

Por otro lado, en eficiencia y escalabilidad es donde podemos encontrar más diferencias entre los métodos. *Monte Carlo*, el método que más episodios necesita (500 000) es el que converge a recompensas más modestas y es muy sensible al valor de ϵ , un equilibrio exploración-explotación mal calibrado degrada significativamente el rendimiento. Continuando con *Q-Learning*, este mejora los resultados anteriores con 20 000 episodios gracias a su actualización paso a paso, que permite aprender de cada transición sin esperar al final del episodio. Finalmente, el *DQN* es el método que menos episodios necesita para alcanzar un buen rendimiento (3 000 episodios). Las variantes *Dueling* y *Double DQN* no han mostrado una mejora reseñable sobre el DQN estándar, resultado razonable teniendo en cuenta el tamaño relativamente pequeño del espacio de observación.

Las principales limitaciones identificadas en el caso de *Monte Carlo* y *Q-Learning* son el uso de la tabla Q , la cual impone una discretización del espacio de estados que introduce pérdida de información, y su tamaño crece exponencialmente conforme aumenta la complejidad del entorno. En el *DQN*, la alta varianza entre episodios sugiere que el agente aún no ha consolidado una política estable ante la aleatoriedad del entorno (colocación de comida y movimiento de obstáculos). Adicionalmente, el número de episodios de entrenamiento del DQN (3 000) puede ser insuficiente para explorar con profundidad el espacio de estados.

Como posibles líneas de mejora, podríamos explorar las siguientes opciones:

- 1) **Aumentar el número de episodios de entrenamiento del DQN** y explorar tasas de aprendizaje más bajas para favorecer una convergencia más estable.
- 2) **Aumentar el espacio de estados con información adicional**, como la distancia exacta a los obstáculos o una representación local del entorno en forma de ventana lo que podría permitir al agente tomar decisiones más informadas.
- 3) **Explorar métodos de política directa como Actor-Critic**, que combinan las ventajas de los métodos basados en valores con el aprendizaje directo de la política π .

Cabe destacar también que el vector de observación, al informar únicamente del peligro inmediato en las celdas adyacentes, limita la capacidad del agente para planificar a largo plazo. En determinadas situaciones, el agente puede tomar decisiones correctas a corto plazo que le lleven a quedarse atrapado más adelante, algo que una representación más completa del entorno podría haber evitado.

6. Referencias

- [1]: [Optimizing Pathfinding in Snake Game using A* and Branch and Bound Algorithm for Safe Apple Collection](#)
- [2]: [Exploration of Reinforcement Learning to SNAKE](#)
- [3]: [A Deep Q-Learning based approach applied to the Snake game](#)